9.3 团队协作与实战案例

> **生成模型**：GPT-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~35k tokens，输出 ~4k tokens（估算，本节）

前两节把多 Agent 的基本零件都讲了：怎么定义多个 Agent，怎么按路由把消息送对，怎么在运行时拉子代理、跨会话通信。现在我们把这些零件拼起来，看几个真正有用的团队协作场景。

这里有个原则先说在前面：多 Agent 最适合的，不是“为了炫技硬拆”，而是那些本来就天然分工的流程。也就是说，如果现实里应该由两个人配合完成，那它往往就适合拆成两个 Agent；如果现实里本来就是同一个人顺手做完的事，硬拆通常只会变慢。

9.3.1 场景一：代码评审 Agent + 部署 Agent

这个场景对开发者最实用。很多项目里，写代码、review 代码、发布上线，本来就不是完全同一种工作。让同一个 Agent 从改代码一路做到账户登录、部署、验收，风险其实不小。

更稳的结构是这样：

- `reviewer` ： 只看变更质量、测试覆盖、风险点。
- `deploy` ： 只负责发布、回滚、环境检查。

为什么这样拆？因为 review 的重点是“判断”，deploy 的重点是“执行”。前者要上下文细、标准严，后者则要权限明确、动作可控。

完整配置示例如下：

```
{
  agents: {
    list: [
      {
        id: "reviewer",
        name: "Code Review Agent",
        model: "anthropic/claude-sonnet-4-5",
        workspace: "~/projects/myapp",
        skills: ["coding-agent", "github"],
        tools: {
          allow: ["read", "sessions_spawn", "sessions_send", "sessions_list", "sessions
          deny: ["write", "edit", "bash"]
        }
      },
      {
        id: "deploy",
        name: "Deployment Agent",
        model: "openai/gpt-5.2",
        workspace: "~/projects/myapp",
        skills: ["memory"],
        tools: {
          allow: ["read", "bash", "sessions_send", "sessions_history"],
          deny: ["write", "edit", "browser"]
        }
      }
    ]
  },
  bindings: [
    {
      agentId: "reviewer",
      match: { channel: "discord", peer: { kind: "group", id: "dev-review-room" } }
    },
    {
      agentId: "deploy",
      match: { channel: "slack", peer: { kind: "group", id: "release-room" } }
    }
  ],
  tools: {
    agentToAgent: {
      enabled: true,
      allow: [
        { from: "reviewer", to: "deploy" },
        { from: "deploy", to: "reviewer" }
```

```
        ]
      }
    }
  }
```

实际流程可以这么跑：开发者把 PR 丢给 `reviewer` ，它先检查改动是否合理、测试是否够、有没有危险脚本；确认通过后，再把“可发布版本 + 风险摘要 + 回滚建议”发给 `deploy` 。 `deploy` 完成上线后，再把环境状态和结果回发。

这里的关键不是让部署 Agent 更聪明，而是让它更克制。它不应该参与大段代码讨论，也不应该自己乱改逻辑。它的职责就是把部署这条线干净利落地走完。

9.3.2 场景二：客服 Agent + 升级处理 Agent

第二个很现实的场景是客服。单一客服 Agent 往往会遇到一个尴尬问题：普通咨询很多，高风险问题很少，但风险一旦来了，处理方式和语气都完全不一样。

所以更好的设计是两层：

- `support` ： 负责接住大多数标准化问题，像查询订单、修改信息、发帮助文档。
- `escalation` ： 只负责退款争议、情绪激动用户、账号安全、投诉升级这类高风险工单。

完整配置可以写成这样：

```
{
  agents: {
    list: [
      {
        id: "support",
        name: "Customer Support Agent",
        model: "openai/gpt-4o-mini",
        skills: ["memory"],
        tools: {
          allow: ["read", "browser", "sessions_send", "sessions_spawn", "sessions_histo
          deny: ["bash", "write", "edit"]
        }
      },
      {
        id: "escalation",
        name: "Escalation Agent",
        model: "openai/gpt-5.2",
        skills: ["memory"],
        tools: {
          allow: ["read", "write", "sessions_send", "sessions_history"],
          deny: ["bash"]
        }
      }
    ]
  },
  bindings: [
    {
      agentId: "support",
      match: { channel: "telegram", accountId: "support-bot" }
    }
  ],
  tools: {
    agentToAgent: {
      enabled: true,
      allow: [
        { from: "support", to: "escalation" },
        { from: "escalation", to: "support" }
      ]
    }
  }
}
```

这种拆法特别有价值，因为它把“效率”和“风险控制”分开了。 `support` 的目标是快，能批量解决的就别升级； `escalation` 的目标是稳，对政策、语气、补偿尺度要更严格。

如果你把两者混在一起，模型很容易在普通咨询里过度谨慎，在高风险工单里又过度随意。角色一分开，风格和权限都更容易守住。

9.3.3 场景三：研究 Agent + 写作 Agent

第三个场景，学生和做知识型工作的开发者都会很常用：查资料和写成文，其实不是一种能力。会查的人不一定会写，会写的人也不一定擅长做事实比对。

所以这条链路特别适合拆成：

- `research` ： 查资料、比对来源、做事实清单。
- `writer` ： 组织结构、写成读者能看懂的文档。

完整配置示例如下：

```
{
  agents: {
    list: [
      {
        id: "research",
        name: "Research Agent",
        model: "google/gemini-2.5-pro",
        skills: ["browser"],
        tools: {
          allow: ["browser", "read", "sessions_send", "sessions_history"],
          deny: ["write", "edit", "bash"]
        }
      },
      {
        id: "writer",
        name: "Writing Agent",
        model: "openai/gpt-5.2",
        workspace: "~/docs",
        skills: ["memory"],
        tools: {
          allow: ["read", "write", "edit", "sessions_send", "sessions_history"],
          deny: ["bash", "browser"]
        }
      }
    ]
  },
  bindings: [
    {
      agentId: "writer",
      match: { channel: "webchat", accountId: "*" }
    }
  ],
  tools: {
    agentToAgent: {
      enabled: true,
      allow: [
        { from: "writer", to: "research" },
        { from: "research", to: "writer" }
      ]
    }
  }
}
```

这个模式的实战价值很高。因为“会查资料”常常意味着上下文里有一堆网页碎片，而“会写文档”需要的是清楚的结构、稳定的术语和克制的表达。两件事拆开之后，writer 接到的是整理后的结论，不会被来源噪音拖着跑。

9.3.4 怎么划边界，决定了多 Agent 是加速还是添乱

看完上面三个场景，你大概已经发现一个规律：边界划得好，多 Agent 就像团队；边界划不好，多 Agent 就像群聊。

有几条经验非常重要。

第一，按职责拆，不要按心情拆。比如“一个负责白天，一个负责晚上”这种分法通常没意义；“一个负责审查，一个负责执行”就很稳。

第二，让每个 Agent 拥有可解释的成功标准。reviewer 的成功标准是发现风险，不是把代码写出来；deploy 的成功标准是发布稳定，不是讨论架构。

第三，工具权限跟职责一起设计。能 review 的不一定能部署，能写文档的不一定能联网查资料。别因为配置方便就全开。

第四，跨 Agent 通信尽量短。发事实、发结论、发待办，不要把一整段啰嗦上下文直接倒给对方。Agent 之间也怕被废话淹没。

第五，保留一个“主负责人”。很多流程可以多 Agent 协作，但最好始终有一个主 Agent 负责最终整合。否则就会出现每个人都说了一点，最后没人对结果负责。

9.3.5 学生和个人开发者怎么落地

你可能会说，这些例子听着像团队，其实我只有一个人。没关系，多 Agent 对个人开发者一样有用。

一个人做项目时，最常见的三种脑力切换就是：写代码、查资料、写说明。单 Agent 会把这三种工作混成连续对话，多 Agent 则等于你给自己搭了三个“专注模式”。同一个人做事，照样可以有岗位分工。

一个很接地气的组合是：

- 平时用 `code` 管仓库。
- 遇到不确定接口时叫 `research` 。
- 准备交作业、写博客、写项目文档时叫 `writer` 。

这比一直对着一个全能 Agent 说“现在你切换成研究模式”“现在你再切换回写代码模式”要稳得多。因为配置里已经把边界固化了，你不用每次都靠 prompt 提醒它“别串戏”。

9.3.6 最后一个建议：多 Agent 要像组织图，不要像技能树

很多人会把多 Agent 配成游戏技能树：今天想到一个功能，就加一个 Agent。结果配着配着，系统里全是名字很酷但职责模糊的角色。

更好的想法是把它当组织图。组织图里的每个岗位都要回答三个问题：它负责什么？它不能做什么？它和谁交接？这三个问题一旦明确，配置自然就会变得简单。

所以，别追求 Agent 数量。追求清楚的边界、稳定的协作链路、可复查的结果。这样多 Agent 才会真的省心。

本节小结

多 Agent 最适合落在那些天然存在分工的流程里：代码评审和部署、客服和升级处理、研究和写作，都是非常典型的例子。配置时最重要的不是把案例抄下来，而是学会一种方法：按职责划边界，按风险配权限，按交接关系开通信。只要这三件事抓住了，你就能把 OpenClaw 从“一个能干很多事的助手”，升级成“一个分工清楚、能互相协作的小团队”。至于多 Agent 底层是怎么解析、怎么维护生命周期、怎么做更深的机制控制，那就是后面第 39 章要继续展开的内容了。